

Mini Report: Multi-Cycle RISC-V Processor

1. Design Overview

This design implements a multi-cycle RISC-V processor in Verilog. The processor executes the same verification program used for the previous single-cycle version, but the instruction execution is split into several FSM-controlled cycles. This allows one shared memory and one ALU to be reused across different stages.

The implementation keeps the reusable module names from the single-cycle design where possible: pc, adder, mux2, mux4, my_reg, my_mem, sign_ext, alu, alu_decoder, main_decoder, controller, and the top-level rv_mc. The multi-cycle design additionally uses mc_reg to implement internal non-architectural registers such as instr_reg, data_reg, rd1_reg, rd2_reg, alu_reg, and PC_OLD_REG.

The shared memory instance is named IMEM so the testbench can still load the program with `$readmemh("programs/program.hex", dut.IMEM.RAM)`. In this multi-cycle design, IMEM.RAM is not instruction-only memory; it is the shared instruction/data memory.

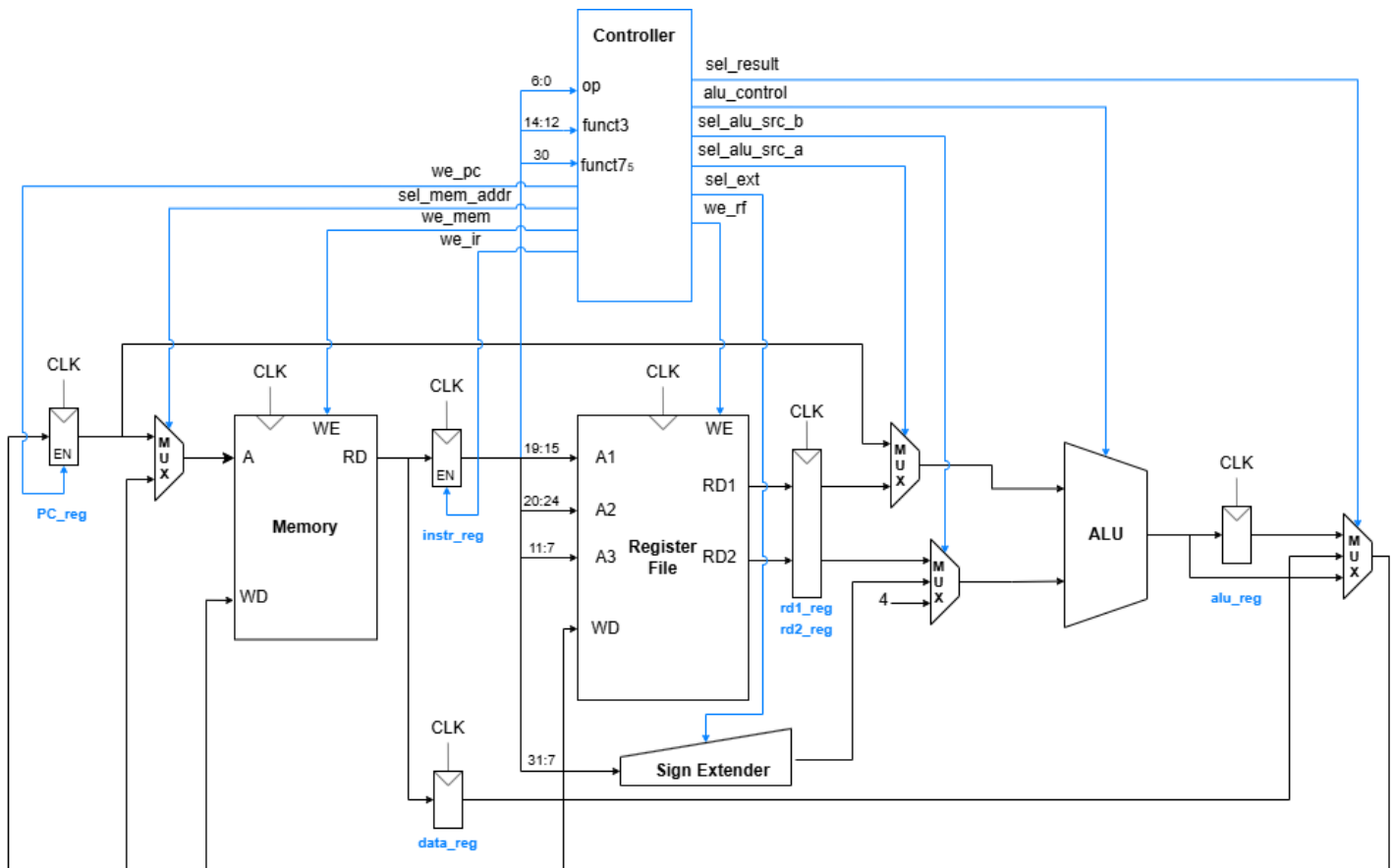


Figure 1: Full multi-cycle system diagram / datapath

2. Datapath and Control Signals

The datapath is organized around one shared memory, one register file, one ALU, and several intermediate registers inserted between major functional blocks. The controller chooses each datapath source by mux control signals.

Signal	Encoding / value	Purpose
sel_mem_addr	0: pc_q; 1: alu_reg_q	Selects memory address for instruction fetch or data access.
sel_alu_src_a	00: pc_q; 01: pc_old_q; 10: rd1_reg_q; 11: 0	Selects ALU input A. pc_old_q is used for branch/jump target calculation.
sel_alu_src_b	00: rd2_reg_q; 01: imm_ext; 10: 4; 11: 0	Selects ALU input B.
sel_result	00: alu_reg_q; 01: data_reg_q; 10: alu_result; 11: imm_ext	Selects write-back/PC-update data. JAL has a special PC path.

we_pc	1 enables PC_reg write	Updates PC.
we_ir	1 enables instr_reg write	Loads a new instruction only in fetch. Also enables PC_OLD_REG.
we_mem	1 enables memory write	Used by sw.
we_rf	1 enables register-file write	Used by lw, R-type, l-type, jal, and lui.

3. FSM Controller

The first-level combinational decoder of the single-cycle version is replaced by an FSM in main_decoder.v. The design uses binary-encoded localparam states. The FSM is Moore-style for most control outputs: outputs mainly depend on the current state, while next-state selection depends on the opcode in S_DECODE. The zero flag affects PC update in S_BEQ.

State	Main control settings	Operation
S0 FETCH	we_ir=1, we_pc=1, sel_mem_addr=0, A=pc_q, B=4, alu_op=ADD, sel_result=alu_result	Fetch instruction into instr_reg, save old PC, compute and write PC+4.
S1 DECODE	A=pc_old_q, B=imm_ext, alu_op=ADD	Decode opcode, read rs1/rs2 into rd1_reg/rd2_reg, and precompute branch/jal target into alu_reg.
S2 EXE_ADDR	A=rd1_reg_q, B=imm_ext, alu_op=ADD	Compute effective address for lw/sw.
S3 MEM_RD	sel_mem_addr=1, sel_result=alu_reg_q	Read data memory at address alu_reg_q into data_reg.
S4 WB_MEM	we_rf=1, sel_result=data_reg_q	Write lw data back to rd.
S5 MEM_WR	we_mem=1, sel_mem_addr=1	Store rd2_reg_q to shared memory at address alu_reg_q.
S6 EXE_R	A=rd1_reg_q, B=rd2_reg_q, alu_op=RTYPE	Execute R-type ALU operation.
S7 WB_ALU	we_rf=1, sel_result=alu_reg_q	Write ALU result back to rd.
S8 BEQ	A=rd1_reg_q, B=rd2_reg_q, alu_op=BRANCH, branch=1, we_pc=zero	Compare rs1 and rs2. If zero=1, write precomputed branch target to PC.
S9 EXE_I	A=rd1_reg_q, B=imm_ext, alu_op=ITYPE	Execute I-type ALU operation.
S10 JAL	we_rf=1, we_pc=1; result=pc_q; pc_next=alu_reg_q	Write PC+4 to rd and jump to precomputed target.
S11 LUI	we_rf=1, sel_result=imm_ext	Write U-type immediate to rd.

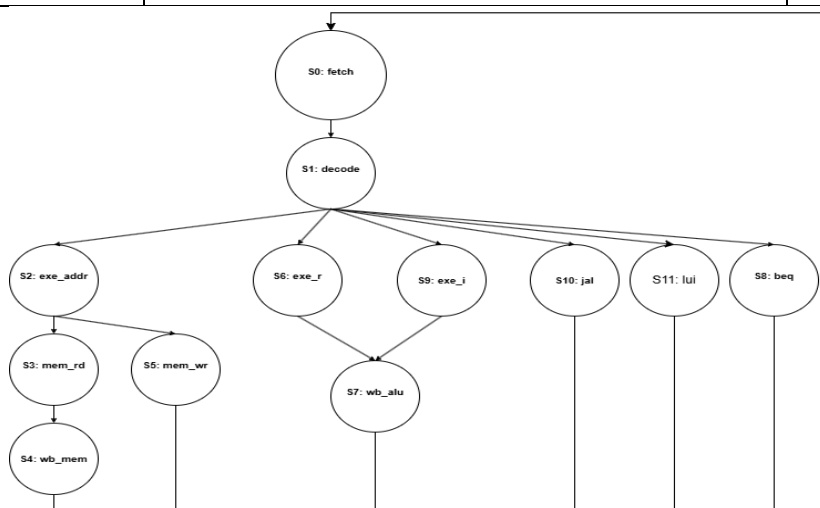


Figure 2: Complete FSM state transition diagram

4. Instruction Flow and Verification Program

Instruction type	FSM path	Cycles
lw	FETCH -> DECODE -> EXE_ADDR -> MEM_RD -> WB_MEM	5
sw	FETCH -> DECODE -> EXE_ADDR -> MEM_WR	4
R-type	FETCH -> DECODE -> EXE_R -> WB_ALU	4
I-type	FETCH -> DECODE -> EXE_I -> WB_ALU	4
beq	FETCH -> DECODE -> BEQ	3
jal	FETCH -> DECODE -> JAL	3
lui	FETCH -> DECODE -> LUI	3

Verification program used in programs/program.hex:

```
addi x1, x0, 5
addi x2, x0, 10
add x3, x1, x2
sw x3, 0(x0)
lw x13, 0(x0)
lui x14, 0x12345
beq x3, x13, label
label:
jal x16, end
end:
addi x18, x0, 42
```

Location	Expected result	Reason
x1	5	addi
x2	10	addi
x3	15	add
IMEM.RAM[0]	15	sw x3, 0(x0) in shared memory
x13	15	lw
x14	0x12345000	lui
x16	0x00000020	jal link address PC+4
x18	42	final addi

5. Performance and Critical Path Analysis

The pseudo component delays are used for report analysis only; they are not implemented with Verilog # delays in synthesizable modules. Each FSM state takes one clock cycle in RTL, and the clock period must be long enough for the slowest register-to-register datapath.

Stage	Representative datapath	Estimated delay
FETCH	PC_reg -> mux -> ALU -> result mux -> PC_reg; PC_reg -> mux -> memory -> instr_reg	max(40+30+30+120+30+40, 40+30+200+40)=310 ps
DECODE	instr_reg -> decoder/register file/sign extender -> rd1/rd2/alu_reg	about 40+20+40+40=140 ps; target path about 40+35+30+30+120+40=295 ps
EXE_ADD R	rd1_reg + imm_ext -> ALU -> alu_reg	40+30+30+35+120+40=295 ps
MEM_RD	alu_reg -> mux -> memory -> data_reg	40+30+200+40=310 ps
EXE_R / EXE_I	operand regs/immediate -> ALU -> alu_reg	about 260-295 ps
WB	data_reg or alu_reg -> result mux -> register file	40+30+40=110 ps

Instruction type	Cycles/instr.	Count	Total cycles
I-type ALU	4	3	12
R-type	4	1	4

sw	4	1	4
lw	5	1	5
lui	3	1	3
beq	3	1	3
jal	3	1	3
Total	-	9	34

Average CPI = total cycles / total instructions = 34 / 9 = 3.78. This manual count should match the simulation waveform if the state sequence is observed from reset release until the last instruction writes x18.

6. Design Discussion

Registers with enable: PC_reg and instr_reg need enable because they must update only in specific FSM states. PC_reg updates in fetch, taken branch, and jal. instr_reg must load a new instruction only during fetch and hold it for the remaining cycles. data_reg, rd1_reg, rd2_reg, and alu_reg can update every cycle because their values are only consumed in the states where they are meaningful.

Why sign extender output is not registered: imm_ext is a combinational function of instr and sel_ext. instr_reg already holds the instruction stable during the instruction execution, so an additional immediate register is not necessary.

One shared memory: Instruction fetch and data access occur in different cycles, so a single memory instance can be reused. The store instruction writes to IMEM.RAM[0] in the test program, replacing the first instruction after it has already been fetched. This is acceptable for this benchmark but real programs should normally keep data away from instruction addresses.

Extensibility: A multi-cycle design can be easier to extend because a new instruction can reuse existing states and functional units or add a small new state. It can also be harder because every new instruction must fit the FSM timing and control-signal schedule.

Bug encountered: A key issue was branch/jal target calculation after PC had already advanced to PC+4. The solution was to add PC_OLD_REG, enabled together with we_ir, so branch and jal targets use the original instruction PC while jal still writes PC+4 as the link address.

Git repo: <https://github.com/linzhi-daphne-Li/Lab-Manual-RISC-V-2.git>